

RELATORIO

ALUNO: CLEITON

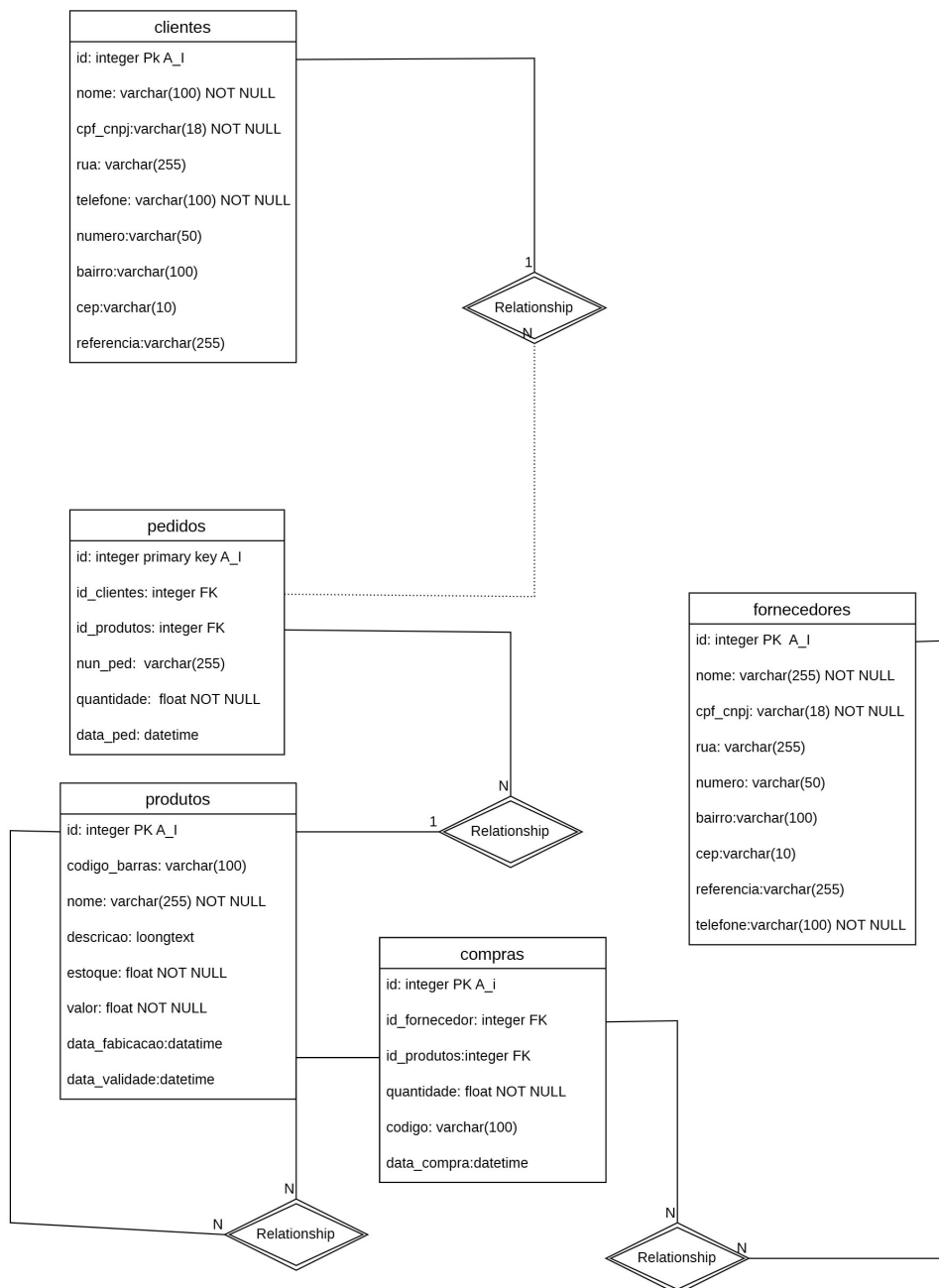
PROFESSOR: LUCAS

Data: 08/10/2016

A escolha do banco de dados relacional surge porque precisamos de segurança, integridade dos dados e organização

TABELAS

O banco de dados possuem as seguintes tabelas



Na tabela clientes ele armazena nome, CPF/CNPJ, rua, telefone, numero, bairro, CEP, referencia

Pedidos armazena : id_clientes (FK), id_produtos(FK), nun_ped, quantidade e data ped

A tabela produtos armazena: código_barras, nome, descrição, estoque, valor, data_fabricação, data_validade

A tabela produtos armazena os seguintes dados: id_fornecedor(FK), id_produtos(FK), quantidade, código e data_compra

A tabela fornecedores armazena : nome, CPF/CNPJ, rua, numero, bairro, CEP, referencia, telefone

S PROUCERES

```
1 CALL sp_update_produtos (101, '1111', NULL, NULL, '500', '400', '2026-08-10', '2030-01-01')
```

```
1 BEGIN
2   UPDATE produtos
3   SET
4     codigo_barras = COALESCE(p_novo_codigo_barras, codigo_barras),
5     nome = COALESCE(p_novo_nome, nome),
6     descricao = COALESCE(p_nova_descricao, descricao),
7     estoque = COALESCE(p_novo_estoque, estoque),
8     valor = COALESCE(p_novo_valor, valor),
9     data_fabricacao = COALESCE(p_nova_data_fabricacao, data_fabricacao),
10    data_validade = COALESCE(p_nova_data_validade, data_validade)
11   WHERE id = p_id;
12 END
```

sp_update_produtos: Esta producer faz um update na tabela produtos

```
1 CALL sp_registrar_compra (78, 39, '50', 'PED-200')
```

```

1 BEGIN
2   INSERT INTO compras (id_fornecedor,
3   id_produtos,
4   quantidade,
5   codigo,
6   data_comprar)
7   VALUES (
8     p_id_fornecedor,
9     p_id_produto,
10    p_quantidade,
11    p_codigo,
12    NOW()
13  );
14 END

```

#	Id	id_fornecedor	id_produtos	quantidade	codigo	data_comprar
1	109	78	39	50	PED-200	2026-10-08 14:46:53

sp_registrar_compra: Este producer faz um INSERT na tabela compras adicionando id_fornecedor, id_produtos, quantidade, código e data_compra

```
CALL sp_compras_por_fornecedor (50, '200-01-01', '2026-12-30')
```

resultado #1 (1r x 5d)

	codigo	produto	quantidade	data_comprar	fornecedor
1	CMP-2007	Mistura para Bolo de Fubá 400g	30	2026-01-25 00:00:00	Distribuidora de Sorvetes Frio Bom

```

1 BEGIN
2   SELECT
3     c.codigo,
4     p.nome AS produto,
5     c.quantidade,
6     c.data_comprar,
7     f.nome AS fornecedor
8   FROM compras c
9   INNER JOIN fornecedores f ON f.id = c.id_fornecedor
10  INNER JOIN produtos p ON p.id = c.id_produtos
11  WHERE c.id_fornecedor = p_id_fornecedor
12  AND c.data_comprar BETWEEN p_data_inicio AND p_data_fim;
13 END

```

sp_compras_por_fornecedor: Busca cada compra que o fornecedor fez

TRIGGER

```

1 BEGIN
2   UPDATE produtos
3   SET estoque = estoque + NEW.quantidade
4   WHERE id = NEW.id_produtos;
5 END

```

Trg_atualiza_estoque_compras: Essa trigger atualiza o estoque compras

```

1 BEGIN
2   UPDATE produtos
3   SET estoque = estoque - NEW.quantidade
4   WHERE id = NEW.id_produtos;
5 END

```

Trg_atualiza_estoque_pedidos: Trigger atualiza estoque pedidos

```

1 BEGIN
2   INSERT INTO tmp_historico_produtos(
3     produto_id,
4     nome_antigo,
5     preço_antigo,
6     estoque_antigo,
7     usuario_alteracao
8   ) VALUES (
9     OLD.id,
10    OLD.nome,
11    OLD.valor,
12    OLD.estoque,
13    USER()
14  );
15 END

```

#	id_historico	produto_id	nome_antigo	preço_antigo	estoque_antigo	data_alteracao	usuario_alteracao
1	1	3	Açúcar Refinado 1kg	4,2	300	2026-10-01 14:44:28	root@localhost
2	2	14	Biscoito Recheado Chocolate 140g	3,5	350	2026-10-01 15:10:55	root@localhost
3	3	101	nao sei	1,0	1	2026-10-01 16:34:15	root@localhost
4	4	101	caneta azul	1,0	1	2026-10-01 16:34:41	root@localhost
5	5	101	caneta azul	2,0	2	2026-10-01 17:14:51	root@localhost
6	6	101	caneta azul	109,0	10	2026-10-08 14:39:42	root@localhost
7	7	39	Sal Refinado 1kg	2,1	500	2026-10-08 14:46:53	root@localhost

Trg_produtos_atualizar: esta tabela insere todos os dados antigos que já foram alterados para a tabela temporária

View

```
select 'c'.nome AS 'nome', count('p'.id) AS 'total_compras' from ('pedidos' 'p' join 'clientes' 'c' on (('c'.id = 'p'.id_clientes')))) group by 'c'.id order by 'total_compras' desc
```

Juliana Mendes	3
Camila Rodrigues	2
João Pedro	2
Fernanda Lima	2
Zé Maria Distribuidora	2
Amanda Farias	2
Caio Cesar	2
Gabriel Alves	2
Lucas Pereira	2
Gisele Batista	2
Diego Carvalho	2
Tatiana Neves	2
Mariana Oliveira	2
Olivia Monteiro	2
Rodrigo Martins	2
Daniel Nogueira	2
Roberto Souza	2
Wagner Duarte	2
Marcos Vinicius	2
Monique Araujo	2
Carlos Eduardo Santos	2
Arthur Aguiar	2
Tech Soluções S/A	2
Isabela Freitas	2
Gustavo Henrique	2
Samuel Correia	2
Comércio de Alimentos Beta ME	2
Larissa Dias	2
Paulo Sergio	2
Empresa Alfa Ltda	2
Yara Medeiros	2
Patricia Rocha	2
Felipe Xavier	2
Leonardo Guimarães	2

View_clientes_com_mais_compras: essa view faz um select nos clientes que tem mais compras